

```
def is_balanced(s):
    stack = []
    bracket_map = {'(': ')', ')': '(', '[': ']', ']': '['}

    for char in s:
        if char in bracket_map.values():
            stack.append(char)
        elif char in bracket_map.keys():
            if stack == [] or stack.pop() != bracket_map[char]:
                return "NO"
        else:
            return "NO"

    return "YES" if not stack else "NO"

if __name__ == "__main__":
    test_cases = [
        "{[()]}",
        "{[()]}",
        "{[[[()]]]}"
    ]

    for s in test_cases:
        print(is_balanced(s))
```

→ YES
NO
YES

```
#Queue using Two Stacks
class MyQueue:
    def __init__(self):
        self.stack_in = []
        self.stack_out = []

    def enqueue(self, value):
        self.stack_in.append(value)

    def dequeue(self):
        self._shift_stacks()
        if self.stack_out:
            return self.stack_out.pop()

    def peek(self):
        self._shift_stacks()
        if self.stack_out:
            return self.stack_out[-1]

    def _shift_stacks(self):
        if not self.stack_out:
            while self.stack_in:
                self.stack_out.append(self.stack_in.pop())

if __name__ == "__main__":
    commands = [
        "1 42",
        "1 14",
        "3",
        "1 28",
        "3",
        "1 60",
        "1 78",
        "2",
        "2",
        "3"
    ]

    queue = MyQueue()
    for command in commands:
        parts = command.split()
        if parts[0] == "1":
            queue.enqueue(int(parts[1]))
        elif parts[0] == "2":
            queue.dequeue()
        elif parts[0] == "3":
```

```
print(queue.peak())
```

```
↔ 42  
42  
28
```

```
#Game of Two Stacks  
def two_stacks(max_sum, a, b):  
    a_sum = 0  
    count = 0  
    i = 0  
    while i < len(a) and a_sum + a[i] <= max_sum:  
        a_sum += a[i]  
        i += 1  
  
    count = i  
    j = 0  
    while j < len(b) and i >= 0:  
        a_sum += b[j]  
        j += 1  
        while a_sum > max_sum and i > 0:  
            i -= 1  
            a_sum -= a[i]  
        if a_sum <= max_sum:  
            count = max(count, i + j)  
  
    return count  
  
if __name__ == "__main__":  
    test_cases = [  
        {  
            "max_sum": 10,  
            "a": [4, 2, 4, 6, 1],  
            "b": [2, 1, 8, 5]  
        }  
    ]  
  
    for case in test_cases:  
        print(two_stacks(case["max_sum"], case["a"], case["b"]))
```

```
↔ 4
```